



Step-by-Step: Use subqueries to aggregate data

This reading provides you with the steps the instructor performs in the following video, [Use subqueries to aggregate data](#). The video teaches you how to aggregate or combine data using subqueries in SQL.

Keep this step-by-step guide open as you watch the video. It can serve as a helpful reference tool if you need additional context or clarification while following the video steps. This is not a graded activity, but you can complete these steps to practice the skills demonstrated in the video.

What you'll need

In order to follow along with the instructor, you will need the `warehouse_orders` dataset uploaded into your project space. From that dataset, you'll need the `warehouse` and `orders` tables. If you haven't already uploaded this data, follow the instructions in the [Upload the warehouse dataset to BigQuery](#) reading.

Objective

The objective of this query is to aggregate the data into a table containing each warehouse's ID, state and alias, and number of orders; as well as the grand total of orders for all warehouses combined; and finally a column that classifies each warehouse by the percentage of grand total orders that it fulfilled: 0–20%, 21–60%, or > 60%.

Note: This activity breaks out the steps into manageable chunks. The final query is only intended to be run at the end. If you try to run the query before reaching the end of this guide you will likely get an error.

Example: Combine and alias the tables

As a refresher, aliasing is when you temporarily name a table or column in your query to make it easier to read and write. To alias the `warehouse` and `orders` tables and join the tables, follow these steps. Remember, these statements require that you enter your unique individual project name or else they won't run. Be sure to substitute your project name in the code wherever you encounter `your-project` written. If you haven't explicitly assigned a project name, BigQuery generates one for you automatically. It typically looks like two words and a number, each separated by a hyphen, for example `august-west-100777`.

Begin with the `FROM` statement a few rows down. Later, you'll return to the top of the query to fill it in.

1. In row 3, enter `FROM your-project.warehouse_orders.warehouse AS Warehouse`
2. In row 4, enter `LEFT JOIN your-project.warehouse_orders.orders AS Orders`
3. In row 5, enter `ON Orders.warehouse_id = Warehouse.warehouse_id`

These statements will combine the two tables (`warehouse` and `orders`) using `warehouse_id` as the common key (the column shared by both tables).

Example: Organize your new table

Use the `GROUP BY` clause in SQL to group rows that have the same values in specified columns into aggregated data, such as sum, count, average, maximum, or minimum,

based on the values in another column. This operation is particularly useful in databases where there is a need to analyze data based on certain criteria.

1. In row 6, enter `GROUP BY`
2. In row 7, enter `Warehouse.warehouse_id,`
3. In row 8, enter `warehouse_name`

Here, the combined table is grouped first by the warehouse ID and then by its name.

Example: Build subquery logic

Now that you have the `FROM` statement and `JOIN`, go back up to the first lines and define the rows to select and operations to perform on them. From the objective, you know you want to return five columns: each warehouse's ID (`warehouse_id`—column 1), state and alias (this info will be combined into a single column: `warehouse_name`—column 2), and number of orders (`number_of_orders`—column 3); as well as the grand total of orders for all warehouses combined (`total_orders`—column 4); and finally a column that classifies each warehouse by the percentage of grand total orders that it fulfilled: 0–20%, 21–60%, or > 60% (`fulfillment_summary`—column 5).

Above everything you've written so far, write:

1. In row 1, enter `SELECT`
2. In row 2, enter `Warehouse.warehouse_id, #` (This is the first column.)
3. In row 3, enter `CONCAT(Warehouse.state, ': ',`
`Warehouse.warehouse_alias) AS warehouse_name, #` (This is the second column. Notice you're concatenating two existing columns into a new one)
4. In row 4, enter `COUNT(Orders.order_id) AS number_of_orders, #` (This is the third column.)
5. In row 5, enter `(SELECT COUNT(*) FROM`
`your-project.warehouse_orders.orders AS Orders) AS`
`total_orders, #` (This is the fourth column.)

To create the final column, you'll need to use a special keyword.

Example: Create categories using `CASE`

Use the `CASE` keyword in SQL to create categories or group data based on specific conditions. This is valuable when dealing with numerical or textual data that needs to be segmented into different groups or categories for analysis, reporting, or visualization purposes.

For the final column, you'll use `CASE` to define which label to apply to each warehouse's fulfillment percentage (the percentage of the grand total of orders that it fulfilled). There will be three conditions, and thus three possible labels: "Fulfilled 0–20% of Orders", "Fulfilled 21–60% of Orders", or "Fulfilled more than 60% of Orders".

1. In row 6, enter `CASE`
2. In row 7, enter `WHEN COUNT(Orders.order_id)/(SELECT COUNT(*) FROM your-project.warehouse_orders.orders AS Orders) <= 0.20` # (This defines the first possible condition.)
3. In row 8, enter `THEN 'Fulfilled 0-20% of Orders'` # (Then defines the label to apply when the first condition is true.)
4. In row 9, enter `WHEN COUNT(Orders.order_id)/(SELECT COUNT(*) FROM your-project.warehouse_orders.orders AS Orders) > 0.20` # (This is the first part of the second condition.)
5. In row 10, enter `AND COUNT(Orders.order_id)/(SELECT COUNT(*) FROM your-project.warehouse_orders.orders AS Orders) <= 0.60` # (This is the second part of the second condition.)
6. In row 11, enter `THEN 'Fulfilled 21-60% of Orders'` # (This defines the label to apply when the second condition is true.)
7. In row 12, enter `ELSE 'Fulfilled more than 60% of Orders'` # (This defines the label to apply when neither of the first two conditions is true.)

8. In row 13, enter `END AS fulfillment_summary #` (The `END` keyword terminates the `CASE` declaration. Then the `AS` keyword indicates what the resulting column should be named.)

Example: Filter using `HAVING`

Use the `HAVING` clause in SQL in combination with the `GROUP BY` clause to filter the results of aggregate functions in a query. While the `WHERE` clause filters individual rows *before* they are grouped, the `HAVING` clause filters groups of rows *after* they have been grouped. To filter out the warehouses that are currently being built (and therefore have no orders), enter the following lines below everything you've written so far:

1. In row 20, enter `HAVING`
2. In row 21, enter `COUNT(Orders.order_id) > 0`

Here is the final query:

```
SELECT
  Warehouse.warehouse_id,
  CONCAT(Warehouse.state, ': ', Warehouse.warehouse_alias) AS
warehouse_name,
  COUNT(Orders.order_id) AS number_of_orders,
  (SELECT COUNT(*) FROM your-project.warehouse_orders.orders AS Orders) AS
total_orders,
  CASE
    WHEN COUNT(Orders.order_id)/(SELECT COUNT(*) FROM
your-project.warehouse_orders.orders AS Orders) <= 0.20
    THEN 'Fulfilled 0-20% of Orders'
    WHEN COUNT(Orders.order_id)/(SELECT COUNT(*) FROM
your-project.warehouse_orders.orders AS Orders) > 0.20
    AND COUNT(Orders.order_id)/(SELECT COUNT(*) FROM
your-project.warehouse_orders.orders AS Orders) <= 0.60
    THEN 'Fulfilled 21-60% of Orders'
    ELSE 'Fulfilled more than 60% of Orders'
  END AS fulfillment_summary
FROM your-project.warehouse_orders.warehouse AS Warehouse
```

```

LEFT JOIN your-project.warehouse_orders.orders AS Orders
ON Orders.warehouse_id = Warehouse.warehouse_id
GROUP BY
    Warehouse.warehouse_id,
    warehouse_name
HAVING
    COUNT(Orders.order_id) > 0

```

Example: Run the new query

It's important to run the new queries that you write to test that they are working correctly.

1. Select the Run button
2. Now, you can identify what percent of our company's total orders are being fulfilled by each warehouse

The query results should be:

Query results					
JOB INFORMATION		RESULTS	CHART	PREVIEW	JSON
		EXECUTION DETAILS		EXECUTION GRAPH	
Row	warehouse_id	warehouse_name	number_of_orders	total_orders	fulfillment_summary
1	1543	KY: Somerset Fulfillment Center	548	9999	Fulfilled 0-20% of Orders
2	9080	KY: Frankfort Fulfillment Center	500	9999	Fulfilled 0-20% of Orders
3	2666	MI: Lansing Fulfillment Center	3178	9999	Fulfilled 21-60% of Orders
4	8118	MI: Ann Arbor Fulfillment Center	3027	9999	Fulfilled 21-60% of Orders
5	4338	TN: Knoxville Fulfillment Center	343	9999	Fulfilled 0-20% of Orders
6	6509	TN: Memphis Fulfillment Center	2403	9999	Fulfilled 21-60% of Orders